ThreatFusion AI: Cyber Threat Intelligence Fusion Using Graph Memory and MITRE ATT&CK

Tanush Bamnote, Vedant Sayare, Jay Gurav, Mukul Suryawanshi, and Anshul Vairagade

Dept. of Computer Science Engineering (Cybersecurity), SVPCET, Nagpur, Maharashtra, India

Project Guide: Dr. Abhishek Pathak, Dept. of CSE (Cybersecurity), SVPCET, Nagpur

***Abstract*—**Analyst burnout in security operations centers is not mainly a volume problem. It is a structure problem. Logs from endpoints, cloud services, and network devices sit in one tool, while the threat intelligence that would make sense of those logs sits in five or six others. An analyst chasing a suspicious IP typically opens VirusTotal, OTX, and a few OSINT feeds manually before getting an answer, and that workflow breaks quickly when the queue has twenty open cases. ThreatFusion AI addresses this at the architecture level. We built a temporal knowledge graph that holds internal telemetry, open-source threat feeds, and MITRE ATT&CK technique mappings in one queryable structure. A team of specialized LLM agents works against that graph to check whether an indicator or behavior pattern has appeared before and which campaign it might belong to. Before any automated containment step executes, a dedicated guardrail agent validates it against policy rules. Low-risk steps go through SOAR automatically; anything higher-impact waits for a human analyst. This paper describes the six-layer architecture, the reasoning behind each design decision, and the evaluation methodology we are running against simulated attack traffic.

Index Terms—Cyber Threat Intelligence, Knowledge Graph, Multi-Agent LLM Systems, MITRE ATT&CK, Security Operations Center, STIX 2.1, Temporal Graph.

I. INTRODUCTION

SOC teams run on SIEM platforms like Wazuh or ELK, ingesting logs from firewalls, endpoints, and cloud services without stopping. Scale is the first obstacle. A 2021 IDC study run with FireEye found that analysts flag false positives in roughly 45 percent of the alerts they actually work through, and a third of respondents admitted to ignoring their queue entirely when volume spiked [12]. Hiring more analysts does not change that ratio. The underlying issue is structural.

Context is the second obstacle. Open-source feeds such as MISP, OpenCTI, AlienVault OTX, and abuse.ch hold indicators that would immediately explain a suspicious connection, but they are not wired into the SIEM. During an active investigation, an analyst copies IP addresses and file hashes into browser tabs one at a time. That workflow does not hold up once the queue grows past a handful of simultaneous cases.

Trying to close that gap with AI creates a third problem. RAG pipelines built on flat vector similarity retrieve text that sounds relevant, but they discard the explicit relationships between entities in the process. That is precisely where large language models fill structural gaps with confident but wrong claims [4], [6]. In a SOC setting, a wrong relationship between an asset and a threat actor can trigger a host isolation that should not happen, or let an analyst dismiss a real compromise.

Campaign invisibility is the fourth obstacle. Indicators of compromise are usually judged one at a time, so a coordinated multi-stage intrusion can go undetected for days because no individual alert ever crossed the severity threshold on its own. ThreatFusion AI addresses all four problems together by giving telemetry and intelligence a shared structural memory and routing specialized LLM agents through that memory instead of through disconnected log lines.

Our contributions are: (1) a six-layer architecture that fuses internal telemetry and external OSINT inside a single STIX 2.1-structured temporal graph; (2) an edge-confidence model that answers whether an indicator has been seen before using a source-weighted numeric score; (3) a guardrail agent placed before execution, with an immutable audit log; and (4) explicit treatment of agent-security risk from attacker-controlled content reaching an LLM, which the related multi-agent SOC systems we surveyed do not address.

II. RELATED WORK

Graph representations of threat intelligence have been around for several years. Pelofske et al. built a Neo4j graph from open-source intelligence text, linking indicators of compromise to MITRE ATT&CK technique IDs so analysts could find connections that keyword search misses [2]. Sun et al. took a narrower approach with SecTKG, a knowledge graph focused on open-source security tooling [3]. Both confirm the value of graph structure in this domain, but neither connects that graph to live internal telemetry or to an active reasoning layer.

GraphRAG has grown considerably as a research area. Zhu et al. proposed KG2RAG, which treats retrieved chunks as fact-level graph nodes rather than isolated text fragments, reporting gains in retrieval precision and answer coherence [4]. Barry et al. applied a similar structure to financial regulatory documents and found that graph-grounded retrieval cut hallucination rates while reducing token cost [5]. Ma et al. showed that even graph-based retrieval drifts when the underlying graph has noisy or incomplete edges [6]. That last finding is why ThreatFusion AI attaches a confidence score to every edge.

Multi-agent systems for cyber defense are more recent. Liu compared centralized, decentralized, and hybrid agent team structures in a tabletop incident-response setting and found that team topology alone changes outcome quality [7]. A 2025 survey argues that multi-agent designs naturally mirror how human incident-response teams split work, reducing the factual drift a single monolithic LLM is prone to [8]. MALCDF puts a four-agent real-time cyber-defense pipeline on record, tested against CICIDS2017 [9]. CyberSleuth applies a MemGPT-style multi-agent design to web-attack forensics [10]. Tellache et al. combine hybrid retrieval with LLM response generation in a CTI-assistant context [11]. Liu and Anwar add RAG directly into a multi-agent incident-response loop in AutoBnB-RAG [15].

ThreatFusion AI draws from all three lines of work. What we add beyond that base is two things we have not seen combined elsewhere: explicit temporal validity windows on every graph edge, and a guardrail agent sitting as a hard gate before any containment action, not as a soft filter afterward.

III. MATERIALS AND METHODS / EXPERIMENTAL SETUP

A. Dataset and Simulation Sources

Our test environment draws from four input categories. For internal telemetry we use Wazuh alert logs, ELK stack event records, Sysmon process and network events, Zeek connection logs, and AWS CloudTrail audit trails. External threat intelligence comes from MISP, OpenCTI, AlienVault OTX, abuse.ch, and OSV. Asset and identity data comes from a synthetic CMDB and an IAM record set we built to represent a realistic mid-size enterprise. Attack traffic is generated with Atomic Red Team scripts, MITRE Caldera operations, and CICIDS dataset samples.

Mixing live-style feeds with labeled simulation traffic gives us two things at once. We get realistic OSINT churn, feed latency, and format variation. We also get ground-truth attack sequences with known technique-to-asset mappings that score automatically, without needing a real breach to produce them. That separation is important for the ablation experiment in Section III-E.

B. Normalization and Schema Mapping

Raw events arrive in different formats: CEF from Wazuh, JSON from CloudTrail, tab-delimited records from Zeek, and STIX bundles from MISP. We map every format onto the Open Cybersecurity Schema Framework (OCSF) [14], a vendor-neutral schema that several major SIEM and cloud vendors have adopted. Normalized records stream over Apache Kafka or Redis Streams into an entity-extraction stage that pulls out IP addresses, file hashes, domain names, user account identifiers, and ATT&CK technique references before any record touches the graph.

We chose OCSF instead of a custom internal format for one practical reason: once every source speaks the same field names, the entity-extraction layer and the graph-write layer do not need separate code paths for each source type. Adding a new feed means writing one OCSF mapping. That tradeoff required upfront work but removed significant complexity from the rest of the pipeline.

C. Knowledge Graph Construction

Extracted entities go into Neo4j under the STIX 2.1 object model, extended with a temporal layer built on the Graphiti agent-memory design [16]. Node types cover assets, user identities, indicators of compromise (IPs, hashes, domains), ATT&CK techniques, and past incident records. Every relationship edge carries three extra

fields: t_start and t_end mark the period during which that link was observed to be active, and a confidence value reflects source reliability weighted by how many independent feeds agree.

When a relationship changes, for instance when a threat actor moves to a new IP address, we do not overwrite the old edge. We stamp it with an invalid_at timestamp and leave it in the graph. Old edges stay queryable because knowing that a relationship was once true has attribution value even after it stops being current. Any query asking whether indicator X is currently tied to threat group Y must match the alert timestamp against the edge validity window, not just match the node.

Feed lag is a practical issue we had to handle directly. OSINT sources like MISP sync on a schedule, not continuously, so an indicator tied to an attack that lands between two sync windows will not have a graph node when the first alert fires. We tag unmatched indicators as context-pending rather than benign. The Correlation Agent re-evaluates them automatically after the next sync completes. Running more frequent syncs shortens that lag window but cannot eliminate it.

D. Agent and Orchestration Setup

Four agents each handle one defined job. The Triage Agent reads incoming OCSF events, removes duplicate alerts, and produces an initial risk score weighted by asset criticality from the CMDB. The Investigation Agent queries graph memory for evidence connected to a specific alert, such as whether the involved IP appeared in recent MISP entries or whether the targeted asset shows up in prior incident nodes. The Correlation Agent looks across graph topology for alerts sharing infrastructure, techniques, or targeted assets, then groups them into campaign nodes. The Guardrail and Policy Agent runs apart from those three. Its only function is to check proposed containment actions against policy rules and blast-radius limits before any SOAR call goes out.

Coordination runs through LangGraph or CrewAI using Model Context Protocol (MCP) tool-calling, giving each agent a consistent interface for graph queries and external API calls. LLM inference can target a cloud endpoint or a locally hosted Llama 3 model via Ollama. The local option exists because some regulated environments cannot route raw log data outside their own network.

E. Evaluation Methodology

Our evaluation has three parts. The first is a campaign-detection accuracy run. We send a set of Atomic Red Team and Caldera attack sequences through the full pipeline and check whether the clusters the Correlation Agent produces match the technique-and-asset sequences in the operation logs. A cluster counts as correct only when it recovers the exact set.

The second part is a controlled ablation. The same alert corpus runs through a flat vector-similarity RAG baseline and through the temporal-graph pipeline. A human annotator counts how often each condition asserts an entity relationship the source data does not support. That directly tests the hallucination argument from Section II. The third part measures latency: median time from alert arrival to a Guardrail Agent decision, compared across both conditions.

IV. RESULTS AND DISCUSSION

A. Architecture Validation

We validated the six-layer pipeline against simulated attack traffic at the time of writing. Test runs covered ATT&CK techniques T1566 (Phishing), T1078 (Valid Accounts), and T1021 (Remote Services), using Atomic Red Team sequences. Each layer handled its designed input and output formats without schema conflicts across Wazuh, Sysmon, and Zeek. Fig. 1 shows the complete system from ingestion through to the interface layer.

OCSF normalization unified all three source formats successfully. Entity extraction pulled IPs, file hashes, and technique references correctly across all tested source types with no per-source logic changes needed after the initial mapping was written.



Fig. 1. ThreatFusion AI six-layer system architecture and guarded execution path.

B. Graph Memory vs. Flat Vector RAG

Our preliminary ablation compared the temporal-graph pipeline against a flat vector-similarity RAG baseline over the same corpus. In early runs, the flat RAG baseline returned at least one unsupported entity relationship in 38 percent of multi-hop queries, where the query traced an indicator across two or more hops. Running the same queries through the temporal-graph pipeline brought that figure down to 7 percent. These numbers come from a small controlled trial. We report them as directional results, not benchmarks.

A specific case illustrated the temporal edge benefit. Three of our simulated attack sequences reused an IP address that had appeared in a MISP feed entry 90 days earlier. The flat RAG baseline linked that alert straight to the old threat actor attribution without flagging the time gap. The temporal-graph pipeline marked that edge as expired and held the attribution as unconfirmed until the next sync completed.

C. Guardrail Agent Performance

In every test run, the Guardrail Agent intercepted all high-impact containment proposals and routed them to the human-in-the-loop console. Low-risk actions, specifically IOC blocking where indicator confidence exceeded 0.85 and blast radius was limited to a single workstation, went through SOAR automatically. No false automatic containments occurred in the controlled test environment.

One test case showed why the pre-execution position matters. The Investigation Agent reached a high-confidence attribution. The Guardrail Agent's policy check flagged it as crossing a blast-radius boundary before any SOAR call was issued. A post-hoc filter working only on the final recommendation would not have had that context and would likely have passed it through.

D. Limitations

All results above come from controlled simulation, not production-scale traffic. Entity resolution accuracy, confidence scoring, and agent-security protections have not been tested at real SOC volumes or under adversarial conditions. The ablation figures in Section IV-B are directional.

Two open problems remain. Graph scaling is one: continuous OSINT ingestion grows node and edge count without a natural bound, and we have no tested pruning or archival policy yet. Inference cost is the other: routing several thousand alerts per day through a cloud LLM endpoint is expensive, and the system does not yet define a routing policy that sends routine triage to a cheaper local model while reserving cloud inference for higher-stakes correlation work.

V. CONCLUSIONS

ThreatFusion AI rests on one argument: the missing piece in AI-assisted SOC tooling is not a more capable model, but a better memory. Holding internal telemetry and external threat feeds together inside a STIX 2.1-structured temporal graph preserves the relational context that flat vector RAG loses when it chunks and embeds unstructured text. Running specialized agents against that graph gives triage, investigation, correlation, and policy enforcement each the narrow focus they need.

Placing the guardrail agent before execution is a small design decision with consequences that are not small. Keeping a human in the loop at the exact point where an automated mistake would be costliest is, in our view, not optional for a system that can trigger host isolation or token revocation.

Four extensions are planned: multi-step automated remediation playbooks through SOAR, GNN-based lateral-movement prediction using the Neo4j graph, bi-directional STIX sharing back to open CTI platforms, and multi-tenant graph isolation for managed security service providers. None of these, and none of the current system, are meant to replace analyst judgment. The goal is a platform that remembers what has already been seen so analysts can focus on what is actually new.

Acknowledgment

The authors thank Dr. Abhishek Pathak, Dept. of Computer Science Engineering (Cybersecurity), SVPCET Nagpur, for guiding the design of this system and for his detailed feedback on the architecture presented here.

References

[1] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, Technical Report, 2018.

[2] E. Pelofske, L. M. Liebrock, and V. Urias, "Cybersecurity threat hunting and vulnerability analysis using a Neo4j graph database of open source intelligence," arXiv:2301.12013, 2023.

[3] S. Sun, C. Huang, T. Wu, and Y. Shen, "SecTKG: A knowledge graph for open-source security tools," Int. J. Intell. Syst., vol. 2023, Art. no. 4464974, 2023.

[4] X. Zhu, Y. Xie, Y. Liu, Y. Li, and W. Hu, "Knowledge graph-guided retrieval augmented generation," arXiv:2502.06864, 2025.

[5] M. Barry et al., "GraphRAG: Leveraging graph-based efficiency to minimize hallucinations in LLM-driven RAG for finance data," in Proc. Workshop Generative AI Knowledge Graphs (GenAIK), 2025, pp. 54-65.

[6] Y. Ma et al., "Toward robust GraphRAG: Mitigating retrieval drift and hallucination from imperfect knowledge graphs," arXiv:2603.14828, 2026.

[7] Z. Liu, "Multi-agent collaboration in incident response with large language models," arXiv:2412.00652, 2024.

[8] "Large language models for security operations centers: A comprehensive survey," arXiv:2509.10858, 2025.

[9] A. Bhardwaj, S. Godika, and Y. Loonker, "MALCDF: A distributed multi-agent LLM framework for real-time cyber defense," arXiv:2512.14846, 2025.

[10] S. Fumero et al., "CyberSleuth: Autonomous blue-team LLM agent for web attack forensics," arXiv:2508.20643, 2025.

[11] A. Tellache et al., "Advancing autonomous incident response: Leveraging LLMs and cyber threat intelligence," arXiv:2508.10677, 2025.

[12] IDC, "The Voice of the Analysts: Improving Security Operations Center Processes Through Adapted Technologies," IDC InfoBrief, sponsored by FireEye, 2021.

[13] OASIS Cyber Threat Intelligence TC, "STIX Version 2.1," OASIS Standard, 2021. [Online]. Available: https://oasis-open.github.io/cti-documentation/

[14] Open Cybersecurity Schema Framework (OCSF) Project. [Online]. Available: https://schema.ocsf.io

[15] Z. Liu and A. Anwar, "AutoBnB-RAG: Enhancing multi-agent incident response with retrieval-augmented generation," arXiv:2508.13118, 2025.

[16] P. Rasmussen et al., "Zep: A temporal knowledge graph architecture for agent memory," arXiv:2501.13956, 2025.

[17] S. Shafee, A. Bessani, and P. M. Ferreira, "False alarms, real damage: Adversarial attacks using LLM-based models on text-based CTI systems," arXiv:2507.06252, 2025.

[18] D. Ceraku, "RAG-H: A RAG-hardened SOC framework for trustworthy threat intelligence and AI defence," in Proc. 25th Eur. Conf. Cyber Warfare and Security (ECCWS), 2025, doi: 10.34190/eccws.25.1.4741.